DISEÑO COMPUTACIONAL · ENTREGABLE 2

Modelo Relacional

Catorce tablas derivadas del MER por el algoritmo formal de mapeo: llaves, acciones referenciales, normalización y mecanismo de cada restricción de integridad.

PROYECTO

AdaptiveLearn

CURSO

DSE-2 · Diseño y Desarrollo de Software Educativo II

METODOLOGÍA

MODESEC · Fase III

FECHA

19 de septiembre de 2026

Metodología: MODESEC — Fase III, Diseño Computacional. Basado en `01-modelo-entidad-relacion.md` — **Quinta revisión, 2026-09-08**, la que responde a la revisión del docente del 2026-09-01.

Qué exige MODESEC aquí, verificado contra su fuente primaria (no asumido): *"Se deberá crear un MER..., un MR (Modelo relacional) basado en el MER y un diccionario de datos para los metadatos (como tipos, tamaños, y otras cualidades de los campos de datos). Estos modelos son de uso generalizado y cuentan con abundante bibliografía."* (`MODESEC_TISE2009.txt` , sección "Modelado del sistema de conocimiento/bases de datos"). Dos consecuencias concretas:

1. MODESEC no inventa su propia notación — delega en la teoría estándar de bases de datos relacionales (Elmasri & Navathe, Silberschatz — "abundante bibliografía"). Este documento sigue el algoritmo formal de mapeo Entidad-Relación → Relacional de esa bibliografía, no una traducción libre.
2. **Tipos exactos, tamaños y "otras cualidades" son del diccionario de datos**, que es el entregable siguiente: `06-diccionario-datos.md` . Este documento define *estructura* —tablas, llaves, acciones referenciales, normalización, restricciones—; el diccionario define *metadatos* de cada campo.

Corrección mayor de esta revisión (2026-09-19)

Este documento estaba **dos revisiones atrasado** respecto al MER. Su encabezado decía "basado en la Tercera revisión (2026-08-26)" y describía 11 tablas, cuando el MER ya iba en la Quinta con 14 entidades: le faltaban justamente **las tres que se agregaron para responder a la revisión del docente** — `experiencia_evidencia` (H-4), `evidencia_dictamen` (H-6) y `presentacion` (H-2/H-3).

El atraso pasó desapercibido por una razón estructural, no por descuido: `verificar_mr.py` comprobaba el MR **contra sí mismo** (DBML contra SQL). Dos artefactos internamente coherentes y a la vez contradictorios con el MER pasaban la prueba sin problema. Esa era la brecha.

Lo que se cambió para que no vuelva a pasar:

Antes	Ahora
MER y MR, dos documentos escritos a mano por separado	`modelo-relacional/datos_mr.py` es **fuente única** del MR; de ahí se generan el diccionario, el DBML y el SQL
Las columnas se listaban aquí Y en el DBML Y en el SQL	Se listan **solo** en `06-diccionario-datos.md` , generado. Este documento conserva lo que es suyo: estructura y razonamiento
Nada comparaba MR contra MER	**`_verificar_mr_mer.py`** : compara `datos_mr.py` contra `diagrama/datos_mer.py` (la fuente del MER) — entidades, atributos, llaves primarias y destino de cada FK

Una columna que exista en el MR y no en el MER ahora **falla la verificación**, a menos que esté documentada con su razón en `EXTRA_JUSTIFICADAS` . Hoy hay exactamente una, y está más abajo.

Regla de construcción

Cada tabla sale de una entidad ya confirmada en `01-modelo-entidad-relacion.md` — no se agrega estructura nueva aquí, salvo donde el MER **delegue explícitamente** una decisión a este documento (ocurre una vez; ver `resultado`).

Lo que sí es trabajo propio de este documento, porque el MER no lo resuelve: las acciones referenciales (`ON DELETE` / `ON UPDATE`) de cada llave foránea, el análisis de normalización, y el mecanismo de aplicación de cada restricción de integridad (RI-1 a RI-11). Donde una de esas decisiones no se pueda fundamentar en un ADR o spec ya escrita, se declara explícitamente como tal — mismo criterio que usa el MER con sus vacíos.

Notación

`Tabla(**columna_pk**, columna, columna_fk)` — llave primaria en **negrita**. Notación de la bibliografía estándar citada arriba, no inventada para este documento.

Convención de nombres (fijada aquí, no implícita)

El MER nombra entidades en español natural — `Ítem`, `Opción`, `Presentación` — con acentos y mayúsculas. Eso es correcto en un modelo conceptual, pero **no puede bajar tal cual a identificadores SQL**: los acentos obligan a comillar cada identificador y las mayúsculas se comportan distinto según el motor. Se fija una única convención:

- **`snake_case`, minúsculas, sin acentos, singular.**

Entidad (MER)	Tabla (MR)	
Docente	`docente`	
Estudiante	`estudiante`	
Experiencia	`experiencia`	
DBA	`dba`	
Evidencia	`evidencia`	
Evidencia_Dictamen	`evidencia_dictamen`	**nueva** (H-6)
Experiencia_Evidencia	`experiencia_evidencia`	**nueva** (H-4)
Inscripción	`inscripcion`	
Ítem	`item`	
Opción	`opcion`	
Presentación	`presentacion`	**nueva** (H-2/H-3)
Respuesta	`respuesta`	
Respuesta_Opción	`respuesta_opcion`	
Resultado	`resultado`	

14 tablas. Reporte no es una tabla — ver su sección más abajo.

Colisión con palabras reservadas: verificada, ninguna. Las candidatas naturales a chocar (`orden`, `grupo`, `estado`, `texto`, `origen`) están en español y no coinciden con `ORDER`, `GROUP`, `CHECK` ni con

ninguna otra reservada del estándar SQL — el idioma del modelo actúa aquí como protección, pero se comprobó en vez de asumirse.

Política de acciones referenciales

La regla general, para no repetirla 17 veces:

- **RESTRICT** hacia una entidad raíz que ningún flujo de este sistema borra (Docente, Estudiante, DBA, Evidencia) — ninguna spec define un "eliminar cuenta" ni un "eliminar DBA"; RESTRICT es un seguro, no una decisión operativa real.
- **RESTRICT** hacia cualquier tabla que registre historial real de uso (Ítem y Opción, como *destino* de un borrado) — proteger el rastro de lo que un estudiante efectivamente vio y respondió importa más que permitir un borrado cómodo.
- **CASCADE** en composición pura, donde la fila hija no significa nada fuera de su dueño y no hay historial que proteger.
- `ON UPDATE CASCADE` en las 17 FK, sin excepción — estándar para llaves sustitutas (nunca cambian, la cláusula no hace nada en la práctica) y necesario de verdad para la única llave natural del modelo, `codigo_dba`, que si el MEN corrige un código alguna vez debe propagarse.

Cambio real de esta revisión en la protección del historial. Antes, la barrera contra borrar un Ítem con datos reales era la FK de `respuesta`. Ahora es la de `presentacion`, y eso es más fuerte: un ítem queda protegido desde que **se le mostró** a alguien, aunque esa persona no lo haya respondido. Es exactamente lo que pide RI-10 del MER ("Ítem y Opción no pueden editarse una vez que existe una Presentación de ese Ítem"), aplicado al borrado a nivel de esquema.

El detalle completo de las 17 FK, con su acción y su razón una por una, está en `06-diccionario-datos.md`, tabla por tabla.

Análisis de normalización

Cada tabla, hasta qué forma normal llega y por qué. Las columnas de cada una están en el diccionario; aquí va el razonamiento, que es lo propio de este documento.

Llave simple, sin dependencias entre atributos no clave → 3FN

`docente`, `estudiante`, `experiencia`, `inscripcion`, `item`, `opcion`, `respuesta`, `dba`.

En todas, cada atributo no clave depende del identificador completo y de nada más. 2FN es trivial (no hay llave compuesta de la que depender parcialmente) y no hay cadenas transitivas.

Tres llegan además a **BCNF por tener una segunda llave candidata**: `docente.correo`, `estudiante.correo` y `experiencia.codigo_invitacion` son `UNIQUE NOT NULL`, y ninguna determina parcialmente a la otra.

Un caso que **parece** violación y no lo es: en `opcion`, las columnas `orden`, `es_correcta` y `grupo` tienen valor válido o nulo según `item.formato` (RI-4). Cada una sigue dependiendo de `id_opcion`; lo que depende de otra tabla es su *dominio válido*, no su determinante. No es un problema de forma normal — es

precisamente el tipo de restricción que un `CHECK` de una sola fila no alcanza a expresar, y por eso figura como "no automática" en la tabla de mecanismos.

Llave compuesta — el análisis sí es necesario

`evidencia` (`codigo_dba`, `numero_evidencia`): ¿`enunciado` depende de las dos columnas o de una sola? De las dos — un `numero_evidencia` repetido entre DBA distintos no identifica la misma evidencia (`numero_evidencia = 1` de `matematicas_8_dba_1` no tiene relación con el `1` de `lenguaje_3_dba_5`). Sin dependencia parcial → 2FN; sin transitiva → 3FN.

`evidencia_dictamen` (`codigo_dba`, `numero_evidencia`): `aplica_formato_cerrado`, `justificacion` y `fecha_dictamen` dependen de qué evidencia se dictaminó, es decir del par completo → 3FN.

`experiencia_evidencia` (`id_experiencia`, `codigo_dba`, `numero_evidencia`): `incluida` depende de las tres — la misma evidencia puede estar incluida en una experiencia y no en otra. Ese hecho es justamente la razón de que esta tabla exista → 3FN.

`presentacion` (`id_inscripcion`, `id_item`): `orden_presentacion` y `fecha_presentacion` dependen del par completo — el mismo ítem se presenta en distinto orden a distintos estudiantes → 3FN.

`respuesta_opcion` (`id_respuesta`, `id_opcion`): `valor_dado` depende del par entero — una misma opción puede tener `valor_dado` distinto según la respuesta en que aparezca (en `ordering` y `matching`, cada estudiante ordena o empareja distinto) → 2FN, y sin transitiva → 3FN.

`resultado` (`id_inscripcion`, `codigo_dba`, `numero_evidencia`): `estado` y `fecha_calculo` dependen de la combinación completa —qué estudiante, qué evidencia— no de una parte → 3FN. Su columna `id_experiencia` merece sección propia, abajo.

La única decisión que el MER delegó a este documento

El MER, al definir `Resultado`, deja escrito:

> *"La experiencia concreta se deduce de `id_inscripcion`; que esa deducción se resuelva con una columna redundante o con una regla, lo decide el Modelo Relacional — es exactamente el tipo de detalle que le corresponde a él, no al MER."*

Es la única delegación explícita del MER, y aquí se resuelve.

El problema. Tras la Quinta revisión, un Resultado ya no mide "la evidencia del MEN en abstracto" sino *la evidencia de esa experiencia* — o sea, apunta a `experiencia_evidencia`, cuya llave tiene **tres** columnas, incluida `id_experiencia`. Pero `resultado` se identifica por `id_inscripcion`, y la inscripción ya determina la experiencia. Hay dos caminos hacia la misma experiencia, y tienen que coincidir.

Las dos salidas posibles, y por qué se eligió esta. Dejarlo como regla externa habría reintroducido exactamente el tipo de restricción que H-4 acababa de eliminar: "dos caminos que un trigger debe obligar a coincidir". La alternativa —una columna redundante— suele ser peor diseño, pero **en este caso concreto no lo es**, porque permite convertir las dos coherencias en estructura:

`resultado` lleva `id_experiencia` y **dos** llaves foráneas compuestas:

FK	Destino	Qué fuerza
`(id_inscripcion, id_experiencia)`	`inscripcion(id_inscripcion, id_experiencia)`	Que la experiencia del resultado sea **la de su propia inscripción**
`(id_experiencia, codigo_dba, numero_evidencia)`	`experiencia_evidencia(...)`	Que la evidencia medida sea una de las que **esa experiencia incluye**

La primera FK apunta a un `UNIQUE(id_inscripcion, id_experiencia)` declarado en `inscripcion` con este único propósito — es trivialmente único (`id_inscripcion` ya es PK), pero hay que declararlo para poder ser destino de una FK.

El resultado: la vieja RI-2 —"la evidencia de un Resultado debe pertenecer al DBA de su Experiencia"— **deja de necesitar trigger**. El motor la rechaza solo. Está probado en `verificar_mr.py` , dos pruebas de violación distintas.

Esta columna es la única del MR que no está en el MER, y por eso está declarada en `EXTRA_JUSTIFICADAS` dentro de `_verificar_mr_mer.py` , con esta razón escrita. Sin esa declaración, la verificación falla.

Un hueco que encontró la construcción de este documento

Nada impide que una `presentacion` empareje una inscripción de la experiencia A con un ítem de la experiencia B. Con `item` colgando de `experiencia_evidencia` e `inscripcion` colgando de `experiencia` , las dos ramas son válidas por separado; lo que ninguna estructura obliga es que lleguen a la misma experiencia.

Se resolvería con el mismo mecanismo que se acaba de usar en `resultado` —llevar `id_experiencia` a `presentacion` y componer las FK—, pero **no se hace aquí**, y la razón es de método: el MER delegó explícitamente la decisión de `resultado` y no dijo nada de `presentacion` . Inventar estructura que el MER no autorizó sería exactamente lo que la regla de construcción de este documento prohíbe.

Queda **declarado como hueco**, para que lo resuelva la próxima revisión del MER, no este documento en silencio. En la práctica el caso es improbable (la aplicación presenta ítems del banco de la propia experiencia del estudiante), pero "improbable" no es "imposible", y esa diferencia es justo la que este proyecto no deja tapada.

Reporte — no es una tabla

Confirmado en el MER: Reporte no persiste, no tiene llave propia, no entra en el conteo de tablas. Se implementa como **vistas de solo lectura** sobre `resultado` (join contra `inscripcion` para "Estado general" agrupado por estudiante; consulta agrupada por `(codigo_dba, numero_evidencia)` para "Estado por evidencia"). No hay DDL de tabla que crear, solo consultas.

Lo que **sí cambió** con la Quinta revisión: la "Vista de estudiantes (aula)" ya es definible. Antes dependía de V-3 (estado de avance de la Inscripción), sin resolver. ADR-022 lo resolvió derivando ese estado de la **ausencia de fila** en `resultado` para alguna evidencia — sin columna ni tabla nueva. Con `presentacion`

existiendo, además se puede distinguir "no ha empezado" de "empezó y no ha terminado", que es lo que el separador "Sin concluir" de `14-spec-vista-interna-estudiantes.md` le promete al docente.

Restricciones de dominio (CHECK)

Los dominios que el modelo declaraba en prosa, bajados a restricción. Todas se verifican mirando **una sola fila**, sin consultar otra tabla — por eso el esquema las aplica solo, y omitirlas sería desaprovechar la garantía más barata del modelo.

Tabla.columna	CHECK	Fuente
`experiencia.estado`	∈ {`Borrador`, `Publicada`}	`04-spec-dashboard.md`
`experiencia.codigo_invitacion`	6 caracteres, sin `0`/`O`/`1`/`I`/`L`, las 6 posiciones	RI-3 · `08-spec-unirse-codigo.md` §2.3
`evidencia.numero_evidencia`	> 0	posición dentro del DBA (MER)
`item.formato`	∈ {`single_choice`, `matching`, `ordering`}	ADR-006
`item.origen`	∈ {`ia`, `editado_docente`}	H-5 · ADR-015
`opcion.grupo`	∈ {`izquierda`, `derecha`} o NULL	ADR-014 (solo `matching`, RI-4)
`presentacion.orden_presentacion`	> 0	RI-8
`respuesta.certeza`	entre 0 y 5	ADR-008 punto 2 (CRI, Hasan et al. 1999)
`resultado.estado`	∈ {`muestra`, `no_muestra`, `no_concluyente`}	ADR-008 punto 4

`item.estado_revision` no lleva CHECK, a propósito. Su vocabulario exacto es **V-7**, sin decidir. Ponerle un dominio cerrado aquí sería fijar por la puerta de atrás una decisión que nadie tomó. Queda `VARCHAR`, y el vacío queda visible.

En MariaDB, 6 de estos 9 se expresan como `ENUM` nativo en vez de `CHECK`; es equivalente en efecto, y así está en el diccionario.

Restricciones de integridad → mecanismo de aplicación

Esta es la parte que un modelo relacional bien hecho no puede saltarse: **no todas las restricciones del MER las aplica el esquema solo.** `PK` / `FK` / `UNIQUE` son automáticas; un `CHECK` de una sola fila solo alcanza para reglas que no necesitan mirar otra tabla. Las que sí necesitan mirar otra tabla quedan marcadas como responsabilidad de la aplicación (o de un trigger). Fingir que un `CHECK` las resuelve sería afirmar más de lo que el mecanismo real sostiene.

Numeración según la Quinta revisión del MER, donde RI-1 y RI-2 desaparecieron al volverse estructura (H-4) y entraron RI-8, RI-9 y RI-10.

RI	Qué exige	Mecanismo	¿Automático?
RI-1	Estudiante + Experiencia único en Inscripción	`UNIQUE(id_estudiante, id_experiencia)`	**Sí**
RI-2	Inscripción + Evidencia único en Resultado	La propia PK compuesta	**Sí**
RI-3	Correos y código de invitación únicos, con formato válido	`UNIQUE` + `CHECK` de dominio	**Sí**
RI-4	Dominio y mínimos de Opción según `item.formato`	Trigger o validación en aplicación (depende de otra tabla, y contar opciones exige agregación)	No
RI-5	`incluida` solo si el dictamen lo permite	Trigger o validación (depende de `evidencia_dictamen`)	No
RI-6	Toda Respuesta selecciona ≥1 Opción **del ítem de su Presentación**	Validación en aplicación al guardar	No
RI-7	Experiencia en Borrador no admite Inscripción	Trigger o validación (depende de `experiencia.estado`)	No
RI-8	`orden_presentacion` único por inscripción y evidencia	`CHECK > 0` es automático; la **unicidad por evidencia** exige join contra `item`	Parcial
RI-9	No exceder el tope de 6 ítems por evidencia (ADR-016)	Validación en aplicación (exige contar)	No
RI-10	Ítem/Opción no editables tras existir Presentación	**El borrado sí es automático** (`RESTRICT` desde `presentacion`); la **edición** sigue siendo responsabilidad de la aplicación	Parcial
RI-11	`respuesta.correcta` y `resultado.estado` coinciden con su recálculo	Disciplina de escritura (ADR-021): se escriben en el momento del cálculo y no se vuelven a tocar	No

Dos cosas mejoraron respecto a la versión anterior de este documento:

1. **Las viejas RI-1 y RI-2 ya no están en esta tabla** — no porque se hayan relajado, sino porque **dejaron de ser reglas**: `item` colgando de `experiencia_evidencia` y las dos FK de `resultado` las convierten en estructura. Eran las dos únicas que exigían comparar dos caminos a través de tres tablas.
2. **RI-10 pasó de "no expresable" a "medio automática".** En la versión anterior era V-2, un vacío declarado: no existía el hecho "este ítem ya se le mostró a alguien", así que la regla no se podía ni enunciar en el esquema. Con `presentacion`, el borrado queda bloqueado por el motor. La edición sigue sin mecanismo de esquema, y eso se dice.

Sobre RI-11, que no es gratis. Con ADR-021 los valores derivados se persisten, así que la coincidencia entre lo guardado y su recálculo **no es automática**: depende de disciplina de escritura. Es un costo real y consciente, a cambio de que un resultado histórico no mute cuando cambia la regla de cálculo. Marcarla como automática sería mentir sobre lo que el mecanismo sostiene. Y hay un límite adicional, heredado del MER: **la regla de parada exacta es V-5, sin definir** — hasta que se defina, RI-11 es verificable en forma pero no en contenido.

Índices

Sin índice, cada `JOIN` por llave foránea y cada verificación referencial recorren la tabla entera. La regla es indexar toda FK — pero **no a ciegas**: una FK cuyas columnas ya son el prefijo izquierdo de un índice existente (la PK o un `UNIQUE`) ya está cubierta, y crear otro sería redundante.

10 índices a crear; 7 de las 17 FK ya están cubiertas por llaves existentes:

FK ya cubierta	Por qué
`evidencia(codigo_dba)`	1.ª columna de su PK compuesta
`evidencia_dictamen(codigo_dba, numero_evidencia)`	es su PK completa
`experiencia_evidencia(id_experiencia)`	1.ª columna de su PK
`inscripcion(id_estudiante)`	1.ª columna del `UNIQUE` de RI-1
`presentacion(id_inscripcion)`	1.ª columna de su PK
`respuesta(id_inscripcion, id_item)`	es el `UNIQUE` declarado
`respuesta_opcion(id_respuesta)`	1.ª columna de su PK

El caso que se escapa a simple vista es `resultado`: sus columnas `(id_experiencia, codigo_dba, numero_evidencia)` están *dentro* de la tabla pero **no son el prefijo izquierdo de su PK** `(id_inscripcion, codigo_dba, numero_evidencia)`, así que esa PK no sirve para resolver la FK hacia `experiencia_evidencia`. Lleva índice propio, y también lo lleva su otra FK compuesta.

El cálculo no se hace a mano: `generar.py` lo deriva de la misma fuente y escribe los `CREATE INDEX` en `mr_esquema.sql`.

Verificación mecánica (no lectura)

Este modelo no se da por bueno releyéndolo. Se baja a artefactos ejecutables y se somete a un motor real.

Archivo	Qué es
`modelo-relacional/datos_mr.py`	**Fuente única.** Tablas, columnas, tipos, FK, unicidad y CHECK.
`modelo-relacional/generar.py`	Genera el diccionario, el DBML y el SQL desde esa fuente.
`modelo-relacional/mr_adaptivelearn.dbml`	Fuente del **diagrama** — se pega en dbdiagram.io.
`modelo-relacional/mr_esquema.sql`	Esquema **ejecutable** (SQLite), para las pruebas.
`modelo-relacional/_verificar_mr_mer.py`	**Nuevo.** MR contra MER.
`modelo-relacional/verificar_mr.py`	Las pruebas contra el motor.

Qué comprueba `_verificar_mr_mer.py` — la brecha que faltaba

Compara `datos_mr.py` contra `diagrama/datos_mer.py`, que es la fuente del MER y de sus seis diagramas Chen. Verifica que las entidades coincidan una a una, que ningún atributo del MER falte en el MR, que

ninguna columna del MR sea ajena al MER (salvo las declaradas con su razón), que las llaves primarias sean las mismas, y que cada FK apunte a una PK o a un `UNIQUE` real. **Con este script, el atraso de dos revisiones que motivó esta corrección habría fallado en la primera corrida.**

Qué comprueba `verificar_mr.py`

- **A. El motor acepta el esquema.** Una FK hacia una columna inexistente o una PK mal declarada hacen fallar el script — no dependen de que alguien las note leyendo.
- **B. 20 pruebas de violación, más 2 en positivo.** Por cada restricción marcada como automática se intenta insertar el dato que la rompe: correo duplicado, `certeza` = 9, código con `0`, `origen` inventado, borrar un ítem ya presentado, una respuesta sin presentación previa, un resultado cuya experiencia no es la de su inscripción. Si el motor lo acepta, la restricción **no existe de verdad**, diga lo que diga este documento. Las dos **en positivo** comprueban lo contrario: que una presentación *sin* respuesta sea válida —el caso "se le mostró y no respondió" que el modelo anterior no podía representar (H-3)— y que el CASCADE limpie lo que debe limpiar.
- **C. DBML contra SQL.** Los dos describen lo mismo y podrían divergir. Se comparan tabla por tabla, columna por columna y FK por FK.

Última corrida: 31 comprobaciones, todas correctas.

Errores que encontró esta verificación, anotados y no escondidos

1. **El MER se contradecía a sí mismo.** RI-11 declaraba `Inscripcion.estado_avance` como atributo derivado y persistido — atributo que **no existe** en la entidad Inscripción ni en la fuente única del MER, y que **no debe existir**: V-3 se resolvió por ADR-022 derivando el estado de avance de la ausencia de fila en `resultado`, "sin columna ni tabla nueva". Residuo de un borrador previo. Corregido en el MER.
2. **El verificador contó mal, no el modelo.** La comparación DBML↔SQL marcó seis tablas con una columna llamada `indexes`: el patrón de búsqueda leía la línea `indexes {` del bloque de PK compuesta como si fuera una columna. Hubo que averiguar **de qué lado** estaba el error antes de tocar nada — igual que en la revisión anterior, y con el mismo resultado: era la herramienta.
3. **Filas de tabla partidas y pipes sin escapar.** El diccionario generado tenía filas cortadas en dos líneas y valores de dominio (`muestra | no_muestra | ...`) cuyos `|` partían las columnas. Dos defectos de presentación, invisibles en el texto fuente y evidentes al renderizar.

Cambios de esta revisión, uno por uno

#	Qué cambió	Origen
1	`experiencia_evidencia`, tabla nueva	H-4
2	`evidencia_dictamen`, tabla nueva; `aplica_formato_cerrado` sale de `evidencia`	H-6
3	`presentacion`, tabla nueva	H-2 / H-3
4	`item`: se retira `orden_sugerido`, entran `origen` y `estado_revision`	ADR-024 / H-5
5	`item`: sus 3 columnas de FK pasan a ser **una sola FK compuesta** hacia `experiencia_evidencia`	H-4

#	Qué cambió	Origen
6	`respuesta`: cuelga de `presentacion` (FK compuesta) y gana `fecha_respuesta`	H-2 / H-3, cierra V-1
7	`resultado`: gana `id_experiencia` y dos FK compuestas; la vieja RI-2 se vuelve estructural	delegación del MER
8	`experiencia`: gana `fecha_publicacion`	Quinta revisión
9	Las columnas dejan de listarse aquí; viven en el diccionario generado	corrección de método
10	Numeración de RI alineada con la Quinta revisión	Quinta revisión

Lo que no cambió y se conserva verbatim: la política de acciones referenciales, el criterio de índices, la convención de nombres, el análisis de normalización de las tablas que no se tocaron, y las decisiones de ADR-021 sobre persistir valores derivados.